

Java企业级开发核心框架：Spring、Spring Boot与MyBatis

第一部分：框架介绍与Spring Boot的实现机制

1. 框架概述

- 什么是框架？
 - 框架是应用程序的骨架。在编程领域，它是通用的、可复用的软件环境，提供特定功能，汇集代码、库和API以支持系统开发。
- 框架的优势
 - **提高开发效率**：事务处理、安全性等通用工作交给框架，开发人员专注业务逻辑。
 - **提高代码规范性和可维护性**：成熟框架有严格的代码规范，保证团队开发风格统一。
 - **提高软件性能**：减少冗余代码（如Spring的IoC解耦，MyBatis的动态SQL）。
- 主流框架及技术定位
 - **Servlet规范**：底层标准（地基），定义HTTP请求接收方式。
 - 与Tomcat服务器关系：

Tomcat监听对应服务端口

Servlet将网络请求分发到对应的处理方法

Servlet将处理结果以json形式返回
 - Tomcat及Spring Boot 的 容器 概念：

Tomcat提供了一个服务器运行环境，

在这里可以处理java的Servlet代码,路由请求

同时Tomcat还为许多网络异常行为提供了默认处理方案

让开发者专注于业务逻辑
 - **Spring**：核心指挥官，解决企业应用复杂性。

通过控制反转和依赖注入减少复杂对象的创建与管理
 - **MyBatis**：档案员，半自动化ORM，负责SQL执行和数据映射。

通过对象关系映射(ORM)将 数据表结构 映射到 对象

支持自行编写sql逻辑,或插件(如 MyBatisX)自动产生常见sql语句

2. Spring Boot如何实现框架及内嵌容器原理

- 什么是Spring Boot？
 - Spring Boot不是替代Spring，而是与Spring紧密结合以提升开发体验的工具。它集成常用第三方库配置，实现“开箱即用”。

Spring 是一种思想,容器,代理模式稍后再说.

Spring Boot 在Web层面可以说是 `Spring MVC + Tomcat` 实现代码和部署一体化(将tomcat作为依赖配置)

此外Spring Boot 支持注解,集成大量第三方库配置,因此简化代码开发

如注解

`@Bean`: 作用于方法创建新对象

`@Component` (及衍生的 `@Service`, `@Repository`): 作用于类创建新对象

`@Autowired`: 注入对象

- **【深化】内嵌容器与微服务多实例原理**

- **内嵌Tomcat机制**: 传统JSP需外部安装Tomcat; Spring Boot将Tomcat封装为依赖包 (`tomcat-embed-core`)。程序执行 `main` 方法时, 在Java进程内部 `new` 一个Tomcat对象并启动。
- **打Fat Jar**: 代码、依赖库、内嵌Tomcat全被打包成一个 `.jar` 文件, 实现 `java -jar` 一键启动。
- **微服务多实例**: 启动多个微服务即启动多个独立Tomcat实例, 通过配置不同 `server.port` 实现物理隔离与独立部署。也可无感替换为Jetty或Undertow。

因此Spring Boot的运行结果是许多的后端接口

- **为什么学习Spring Boot?**

- 简化编码、简化配置、简化部署、简化监控。它是微服务趋势下的核心技术。

第二部分：从JSP到Spring MVC再到Spring Boot的演进

3. 传统Java EE开发的弊端 (JSP+Servlet时代)

- **可维护性差**: 分层不清晰, 业务逻辑难以分离。
- **代码重用性低**: 重复开发底层功能, 易出漏洞。
- **原生Servlet的痛点**: 每个请求需手动编写 `doGet/doPost`, 手动解析参数, 路由配置繁琐。

4. Spring MVC: Web层的核心调度框架

- **从Servlet到Spring MVC的演进**
 - **Servlet的映射规则**: 基于代码逻辑 (`@webServlet`), 与物理文件结构无关。
 - **Spring MVC的解法**: 提供核心 `DispatcherServlet` (前端控制器) 拦截所有请求, 通过注解化开发将URL映射到普通Java方法。
- **核心组件 (“高级预制板”)**
 - **DispatcherServlet (总包工头)**: 统一分发请求。
 - **HandlerMapping (任务分配簿)**: 维护URL与方法映射关系。
 - **HandlerAdapter (全能翻译官)**: 解析参数, 绑定到Java方法, 并执行方法。
 - **HttpMessageConverter (数据打包机器)**: 将对象序列化为JSON, 或反序列化JSON为对象。

5. 传统MVC到现代Spring Boot的架构转变

- **传统MVC时代**：Model、View(JSP)、Controller全部在后端。后端渲染HTML发给浏览器（甚至需要拼接字符串）。
- **Spring Boot时代**：基于Spring MVC，但通过自动配置和内嵌容器，极大地简化了繁琐的XML配置和部署流程，使其成为现代Web开发的标准底座。

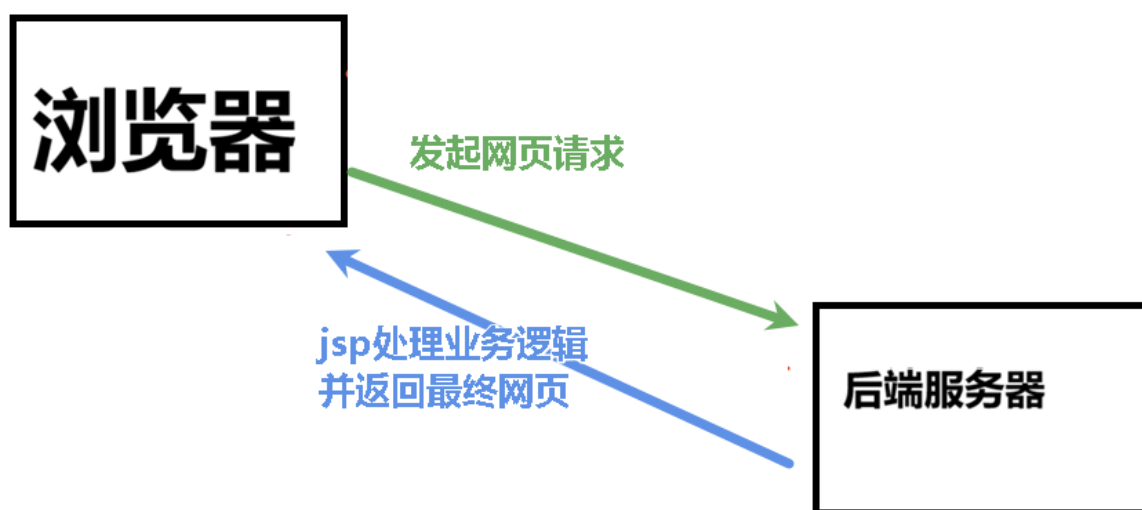
第三部分：Spring Boot、后端接口与前后端分离机制

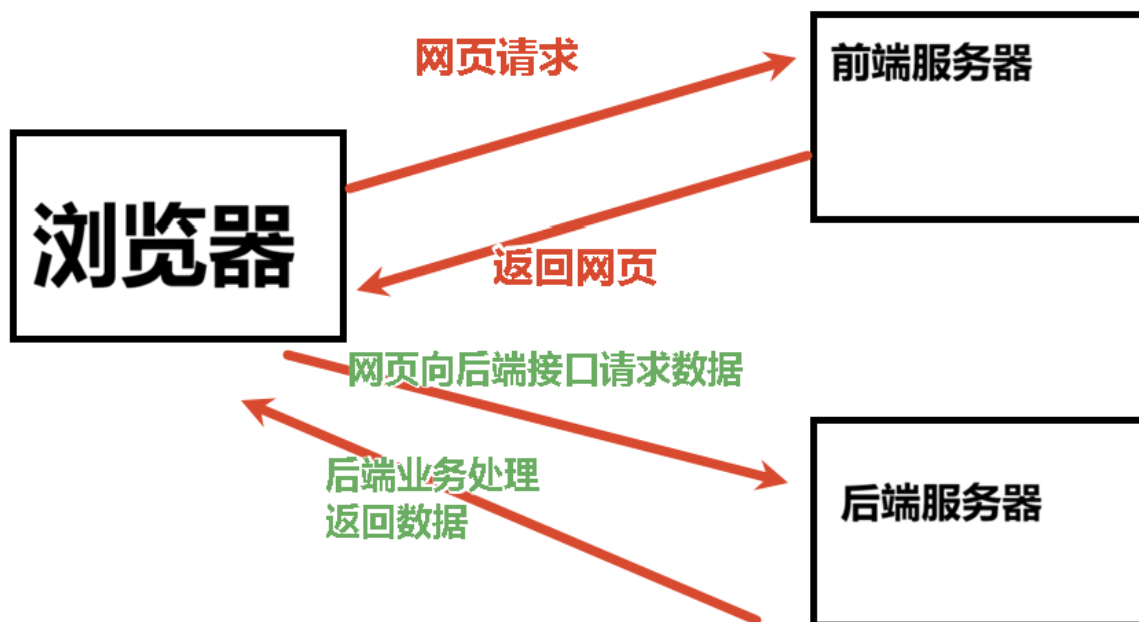
6. MVC架构模式与前后端分离演进

- **现代前后端分离时代**
 - **View的剥离**：View不再由后端渲染，交由前端框架（React/Vue）。前端打包生成的静态文件由Nginx提供访问。
 - **角色分工**
 - **React与Nginx**：React打包为静态文件，由Nginx（文件仓库管理员）负责物理映射，提供静态资源服务。

即前端工作流程
 - **Spring Boot**：退化为纯粹的API数据提供者（加工车间），运行在Tomcat中，执行Java代码，生成JSON数据。

后端工作流程
- **交互流程**：前端通过Ajax请求后端接口，获取JSON数据并在浏览器端自行渲染。





7. Spring Boot核心功能实战（接口开发）

- **JSON数据封装**: 使用 `@RestController` (组合 `@Controller` 和 `@ResponseBody`) , 默认使用 Jackson(库)解析。支持统一返回结构封装 (如 `JsonResult`) 。这是后端分离架构下Spring Boot最核心的产出。
- **MVC支持注解**: `@RequestMapping` 、 `@PathVariable` 、 `@RequestParam` 、 `@RequestBody` 。
- **全局异常处理**: 使用 `@ControllerAdvice` 和 `@ExceptionHandler` 拦截异常, 返回统一JSON结构。
- **AOP切面处理**: 通过 `@Aspect` 定义切面, 实现日志记录、权限校验等解耦操作。
- **拦截器**: 实现 `HandlerInterceptor` 接口, 通过 `WebMvcConfigurer` 注册拦截器。

第四部分：Spring、Spring Boot、MyBatis配置详解

8. Spring框架基础配置

- **核心接口**: `BeanFactory` (基本容器) 、 `ApplicationContext` (丰富特性, 如 `ClassPathXmlApplicationContext`) 。
- **Bean的配置与装配**
 - **XML配置**: 根元素 `<beans>` , 常用属性包括 `id` 、 `class` 、 `scope` 。
 - 如 `<bean id="userA" class="com.example.User" />`
 - **注入方式**:
 - 属性setter方法注入 (`<property>`) 。
 - 构造方法注入 (`<constructor-arg>`) 。
 - 注解方式注入: `@Component` 、 `@Service` 、 `@Autowired` 、 `@Resource` 等。
- **Bean的生命周期**

- `singleton`：容器管理完整生命周期。每个Spring IoC容器中同一个Bean定义只有一个实例
- `prototype`：容器仅负责创建。`prototype` 作用域的 Bean 在**每次被请求时**（无论是通过 `getBean()` 还是依赖注入）都会创建一个全新的实例
- 监控注解：`@PostConstruct`、`@PreDestroy`。

9. Spring Boot配置与属性管理

- **简化配置**：摒弃繁琐XML，采用Java Config（`@Configuration`、`@Bean`）和 `application.yml`。
- **属性配置**：
 - `@Value("${key}")`：获取单个配置。
 - `@ConfigurationProperties`：批量绑定配置到实体类。
 - 支持多环境配置（`application-dev.yml` / `application-pro.yml`）。

10. MyBatis配置与集成

- **MyBatis概述**：半自动化ORM，解决JDBC硬编码问题。
- **MyBatis环境搭建（传统方式）**
 - 导入JAR包（或Maven坐标）。
 - 编写实体类与Mapper映射文件（`Mapper.xml`）。
 - 编写核心配置文件（`mybatis-config.xml`）：配置环境、数据源、Mapper路径。
- **Spring Boot集成MyBatis**
 - **依赖导入**：`mybatis-spring-boot-starter`、`mysql-connector-java`。
 - **配置文件**：在 `application.yml` 中配置数据源及MyBatis属性（如驼峰命名开启）。
 - **整合方式**：
 - 基于XML：添加 `@Mapper` 或 `@MapperScan`，编写XML映射文件。
 - 基于注解：使用 `@Select`、`@Insert` 等注解直接写在Mapper接口上。

11. 其他进阶配置

- **事务管理**：使用 `@Transactional` 注解（注意默认回滚`RuntimeException`）。
- **模板引擎**：集成Thymeleaf（注：前后端分离主流下使用频率降低）。

其他

Spring依赖注入(Dependency Injection)和控制反转(IoC)

- 依赖注入
 - 调用者如 `UserService` 只声明“我需要一个 `UserDao`”，然后由 Spring 容器在背后默默地把现成的 `UserDao` 对象塞给它。
- 控制反转
 - **如何创建、创建哪个实例**由容器决定。这就是“控制权”的**反转**。

```
// IoC 模式 - 声明依赖，由容器注入
@Service
public class UserService {
    private final UserDao userDao;

    @Autowired // 构造器注入，Spring 4.3+ 可省略
    public UserService(UserDao userDao) { // 声明需要 UserDao
        this.userDao = userDao; // 容器注入实例
    }

    public void saveUser() {
        userDao.save();
    }
}
```

mybatis在SpringBoot中使用示例

目录结构

```
mybatis-demo
├── pom.xml # Maven 依赖及项目配置文件
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── example
│   │   │   │   │   ├── demo
│   │   │   │   │   │   ├── MybatisDemoApplication.java # SpringBoot 启动类
│   │   │   │   │   │   ├── controller # 控制器层（负责处理 HTTP 请求）
│   │   │   │   │   │   │   ├── UserController.java
│   │   │   │   │   │   ├── service # 服务层（业务逻辑）
│   │   │   │   │   │   │   ├── UserService.java
│   │   │   │   │   │   ├── mapper # 数据访问层（MyBatis Mapper）
│   │   │   │   │   │   │   ├── UserMapper.java
│   │   │   │   │   │   ├── entity # 实体类层（POJO）
│   │   │   │   │   │   │   ├── User.java
│   │   │   └── resources
│   │   │       ├── application.yml # SpringBoot 核心配置文件（数据源、MyBatis配置）
│   │   │       └── mapper # MyBatis XML 映射文件目录
│   │   │           └── UserMapper.xml # 对应 UserMapper 接口的 SQL 语句
│   └── test # 测试代码目录
```

pom.xml

```
<dependencies>
  <!-- Spring Boot Starter -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- MyBatis Spring Boot Starter -->
  <dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
    <version>2.2.2</version>
  </dependency>

  <!-- MySQL Driver -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Lombok -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/test_db?useSSL=false&serverTimezone=UTC
    username: root
    password: yourpassword
    driver-class-name: com.mysql.cj.jdbc.Driver

  mybatis:
    mapper-locations: classpath:mapper/*.xml # mapper文件位置
    type-aliases-package: com.example.demo.entity # 实体类包路径
    configuration:
      map-underscore-to-camel-case: true # 驼峰命名转换
```

实体类

```
package com.example.demo.entity;

import lombok.Data;

@Data
public class User {
    private Long id;
    private String username;
    private String password;
    private Integer age;
    private String email;
}
```

Mapper接口

```
package com.example.demo.mapper;

import com.example.demo.entity.User;
import org.apache.ibatis.annotations.Mapper;
import java.util.List;

@Mapper
public interface UserMapper {
    List<User> findAll();

    User findById(Long id);

    int insert(User user);

    int update(User user);

    int delete(Long id);
}
```

Mapper XML文件

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.example.demo.mapper.UserMapper">

    <resultMap id="UserResultMap" type="User">
        <id column="id" property="id"/>
        <result column="username" property="username"/>
        <result column="password" property="password"/>
        <result column="age" property="age"/>
        <result column="email" property="email"/>
    </resultMap>

    <select id="findAll" resultMap="UserResultMap">
        SELECT * FROM user
    </select>

    <select id="findById" parameterType="long" resultMap="UserResultMap">
        SELECT * FROM user WHERE id = #{id}
    </select>

    <insert id="insert" parameterType="User" useGeneratedKeys="true" keyProperty="id">
        INSERT INTO user (username, password, age, email)
        VALUES (#{username}, #{password}, #{age}, #{email})
    </insert>

    <update id="update" parameterType="User">
        UPDATE user
        <set>
            <if test="username != null">username = #{username},</if>
            <if test="password != null">password = #{password},</if>
            <if test="age != null">age = #{age},</if>
            <if test="email != null">email = #{email}</if>
        </set>
        WHERE id = #{id}
    </update>

    <delete id="delete" parameterType="long">
        DELETE FROM user WHERE id = #{id}
    </delete>
</mapper>

```

Service 层

```
package com.example.demo.service;

import com.example.demo.entity.User;
import com.example.demo.mapper.UserMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {

    @Autowired
    private UserMapper userMapper;

    public List<User> findAll() {
        return userMapper.findAll();
    }

    public User findById(Long id) {
        return userMapper.findById(id);
    }

    public int save(User user) {
        return userMapper.insert(user);
    }

    public int update(User user) {
        return userMapper.update(user);
    }

    public int delete(Long id) {
        return userMapper.delete(id);
    }
}
```

Controller

```
package com.example.demo.controller;

import com.example.demo.entity.User;
import com.example.demo.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<User> findAll() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public User findById(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping
    public String save(@RequestBody User user) {
        int result = userService.save(user);
        return result > 0 ? "保存成功" : "保存失败";
    }

    @PutMapping
    public String update(@RequestBody User user) {
        int result = userService.update(user);
        return result > 0 ? "更新成功" : "更新失败";
    }

    @DeleteMapping("/{id}")
    public String delete(@PathVariable Long id) {
        int result = userService.delete(id);
        return result > 0 ? "删除成功" : "删除失败";
    }
}
```

启动类

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MybatisDemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(MybatisDemoApplication.class, args);
    }
}
```

启动应用后，可以使用以下API进行测试：

- 获取所有用户：GET <http://localhost:8080/users>
- 根据ID获取用户：GET <http://localhost:8080/users/{id}>
- 添加用户：POST <http://localhost:8080/users> (JSON body)
- 更新用户：PUT <http://localhost:8080/users> (JSON body)
- 删除用户：DELETE <http://localhost:8080/users/{id}>

1. Entity（实体 - 领域层/数据层）

- 职责：代表业务领域的核心概念，包含业务行为和状态。它通常具有唯一标识（ID）。
- 特点：Entity 是“有状态”的，它的属性修改应该通过业务方法（如 `user.changePassword()`）而非简单的 `setter`。

2. DTO（Data Transfer Object - 应用层/接口层）

- 职责：用于进程间通信（如从 Controller 返回给前端，或微服务间调用）。
- 特点：纯数据载体，绝对不应包含业务逻辑。它是为了裁剪、聚合 Entity 数据以满足特定接口需求而存在的。

3. VO（View Object - 表现层）

- 职责：用于页面展示。
- 特点：通常包含前端渲染所需的特定格式（如把状态码 `0` 转化为字符串 `"禁用"`），或者组合多个 DTO 的数据。*（注：在某些框架中 VO 和 DTO 概念常混用，但在严谨架构中职责如上）。*

正确做法（显式转化）：

java

复制

```
1 // 1. 定义纯粹的 DTO
2 public class UserDTO {
3     private Long id;
4     private String username;
5     private String statusText;
6     // 无 passwordHash, 无 isDirty
7 }
8
9 // 2. Controller 显式转化
10 @GetMapping("/user/{id}")
11 public UserDTO getUser(@PathVariable Long id) {
12     UserEntity user = userService.findById(id);
13
14     // 显式转化：开发者清晰地知道这里在做什么
15     UserDTO dto = new UserDTO();
16     dto.setId(user.getId());
17     dto.setUsername(user.getUsername());
18     dto.setStatusText(user.getStatus() == 1 ? "启用" : "禁用");
19     return dto;
20 }
```

mybatis plus函数实现sql

CGLIB（Code Generation Library）是一个强大的、高性能的**代码生成库**，它可以在运行期扩展Java类与实现Java接口。其核心作用是**为没有实现接口的类提供动态代理功能**，是JDK动态代理的重要补充。

· 答案：

B

详细解析：

```
1 // useMemo: 缓存计算结果
2 const result = useMemo(() => computeExpensiveValue(a, b), [a, b]);
3 // result 是 computeExpensiveValue 的返回值
4
5 // useCallback: 缓存函数本身
6 const handleClick = useCallback(() => {
7     doSomething(a, b);
8 }, [a, b]);
9 // handleClick 是一个函数
```

useCallback = useMemo 的特殊版本，专门缓存函数。

使用场景：

- 把函数传给 memo 子组件
- 避免子组件不必要的重渲染

题目 20: Link vs a 标签

问题: React Router 中 Link 和 a 标签的区别?

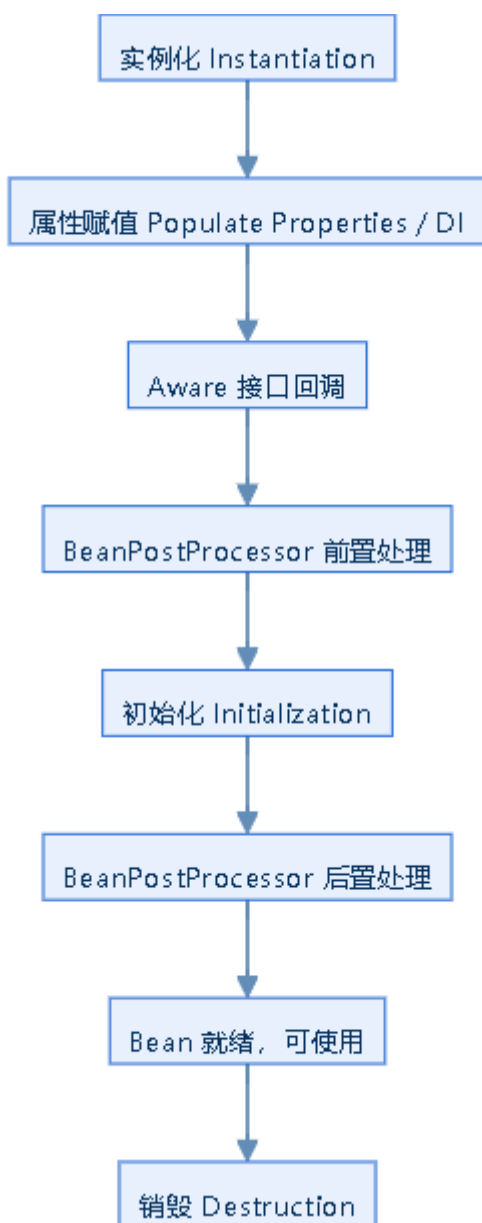
选项:

- A. 没有区别

Link 是不刷新页面内部跳转

```
9  const theme = useContext(ThemeContext);
```

创建 Bean 生命周期



你可以主动获取 Spring 容器中的其他 Bean 或资源，实现更复杂的逻辑。

- **作用：**在这个方法中，你可以对即将初始化的 Bean 进行“前置”处理，例如修改 Bean 的属性、包装 Bean（代理模式）等。**这是 AOP 实现的关键步骤之一。**

你可以对即将初始化的 Bean 进行“前置”处理，例如修改 Bean 的属性、包装 Bean（代理模式）等。**这是 AOP 实现的关键步骤之一。**

Spring Boot 通过以下几个关键技术来实现这一理念：

1. 自动配置 (Auto-configuration)

这是“约定大于配置”最核心的体现。Spring Boot 会根据你添加的依赖（通过 Maven/Gradle 的 “Starter”），自动为你配置相应的功能。

举个例子：

- **传统 Spring：**如果你想使用 JPA（Java Persistence API）来操作数据库，你需要手动在 XML 或 Java 配置类中定义 `DataSource`、`EntityManagerFactory`、`TransactionManager` 等 Bean。
- **Spring Boot：**你只需要在 `pom.xml` 中添加 `spring-boot-starter-data-jpa` 依赖。Spring Boot 会自动检测到这个依赖，并根据默认的“约定”（例如，数据源 URL、用户名、密码通常在 `application.properties` 中配置），为你创建并配置好所有必要的 Bean。
- 当你敲下这个七合一注解时，底层发生了这些与你之前提问息息相关的事情：
 1. **代理模式启动：** `@SpringBootConfiguration` 让 Spring 容器开始用 CGLIB 代理你的配置类。
 2. **工厂模式运转：** `@EnableAutoConfiguration` 和各种 `@Import` 开启了内部的大量 Bean 工厂，批量生产组件。
 3. **三级缓存就绪：** `@ComponentScan` 扫描出的业务 Bean 开始实例化，IoC 容器（一级、二级、三级缓存）全部准备就绪，随时应对可能发生的循环依赖。如果遇到构造器循环依赖，你还需要配合 `@Lazy` 注解来生成代理对象延迟注入。

```

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
// 以下是核心的 7 个注解
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = { ... })
@ConfigurationPropertiesScan
@Import(RuntimeHintsRegistrar.class)
@EnableConfigurationProperties
@AutoConfigurationPackage
public @interface SpringBootApplication { ... }

```

Spring 定义了一组事务相关的行为，我们可以把它们分为三大类来记忆。

二、七种传播行为详解

假设当前存在一个事务 T1，方法 B 被调用，方法 B 配置了不同的传播行为：

第一类：支持当前事务（如果当前有事务，就加入）

1. REQUIRED（必须）—— 默认行为，最常用

- 当前有事务 (T1)：方法 B 加入 T1，在 T1 内部执行。如果 B 抛出异常，T1 整体回滚。
- 当前无事务：方法 B 自己新开一个事务执行。
- 场景：大部分业务逻辑。比如“下单”方法调用“扣库存”方法，扣库存必须在下单的事务里，要么一起成功，要么一起失败。

2. SUPPORTS（支持）

- 当前有事务 (T1)：方法 B 加入 T1 执行。
- 当前无事务：方法 B 以非事务方式执行。
- 场景：一些查询方法。有事务就保证隔离性，没事务也能正常跑。

3. MANDATORY（强制）

当我们在一个事务方法 (A) 中，调用了另一个事务方法 (B) 时，Spring 允许我们通过

`@Transactional` 的 `propagation` 属性来定义 **方法 B 应该如何对待事务**。